

EXP.NO 4 QUERYING LEDGER DATA WITH HYPERLEDGER FABRIC**AIM**

To retrieve and verify asset information from the Hyperledger Fabric blockchain ledger using JavaScript chaincode and the world state database.

PROCEDURE

Step 1: Open the Hyperledger Fabric test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric network and create the application channel.

```
./network.sh up createChannel -ca
```

Step 3: Deploy the JavaScript Asset Transfer chaincode.

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-  
basic/chaincode-javascript
```

Step 4: Initialize the ledger with sample assets.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c  
'{"function":"InitLedger","Args":[]}'
```

Step 5: Retrieve all assets stored in the ledger.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

Step 6: Read the details of asset1.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset1"]}'
```

Step 7: Create a new asset named asset20.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c  
'{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}'
```

Step 8: Query a non-existing asset to verify error handling.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset50"]}'
```

Step 9: Stop the Fabric network.

```
./network.sh down
```

OUTPUT:

```
Jaishree@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Creating channel 'mychannel'... done
Network Started Successfully

Jaishree@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascript -
ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Chaincode committed successfully
```

```
Jaishree@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o
localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C
mychannel -n basic
-c '{"function":"InitLedger","Args":[]}'
Ledger Initialized Successfully
```

```
Jaishree@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n
basic
-c '{"Args":["ReadAsset","asset1"]}'
{
  "ID": "asset1",
  "Color": "blue",
  "Size": 5,
  "Owner": "Tomoko",
  "AppraisedValue": 300
}
```

```
Jaishree@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o
localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C
mychannel -n basic
-c '{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}'
Asset asset20 created successfully
```

```
Jaishree@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n
basic
-c '{"Args":["ReadAsset","asset20"]}'
{
  "ID": "asset20",
  "Color": "Green",
  "Size": 8,
  "Owner": "Arun",
  "AppraisedValue": 1500
}
```

```
Jaishree@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n
basic
-c '{"Args":["ReadAsset","asset50"]}'
Error: The asset asset50 does not exist.
```

RESULT:

The ledger data stored in the Hyperledger Fabric blockchain network was successfully queried using JavaScript chaincode. Asset records were retrieved from the world state database, specific asset details were verified, and newly created assets were successfully queried, demonstrating efficient data retrieval and verification in a permissioned blockchain environment.